Projet Usine 4.0

Application au deep learning, ING3 option IA, CY Tech. Enseignant : Franklin Brasa.

Le besoin

Une aciérie contrôle aujourd'hui ses tôles à l'œil : des opérateurs regardent défiler les tôles et signalent les défauts de surface. Le contrôle est fatigant, irrégulier d'une équipe à l'autre, et certains défauts passent. La direction se demande si une caméra et un modèle de vision pourraient faire ce travail, ou au moins aider les opérateurs.

Elle vous confie l'étude. Elle dispose de photos de défauts déjà annotées (le jeu NEU-DET : six types de défauts, avec leur position sur l'image), mais certains défauts sont rares sur la ligne : pour ceux-là, on n'aura que quelques images. Le système tournera sur un PC de bureau installé sur la ligne, sans carte graphique, comparable aux machines de l'école. La caméra prend une image par seconde : le système doit donc traiter une image en moins d'une seconde. La direction veut savoir, à la fin de l'étude, s'il faut lancer le projet, et à quelles conditions.

Votre mission

Concevoir, évaluer et défendre un système d'inspection des tôles, puis faire une recommandation à l'usine.

Le système doit au minimum :

- reconnaître le type d'un défaut et le localiser sur la tôle ;
- tenir la cadence d'une image par seconde sur le PC de la ligne : mesurez son temps de traitement et appliquez au moins une optimisation vue au Jour 6 (quantification, export ONNX ou distillation, par exemple), en mesurant ce qu'elle fait gagner et perdre ;
- être accompagné d'un plan de suivi une fois en service : quelles grandeurs surveiller, et à partir de quand s'inquiéter.

Vous ajoutez un volet au choix :

- les défauts rares : comment traiter un type de défaut dont on n'a qu'une quinzaine d'images ? Vous choisissez le type de défaut et le réduisez vous-mêmes à 15 images dans l'entraînement ;
- la maintenance : l'usine voudrait aussi anticiper les pannes de ses pompes à partir de leurs capteurs (jeu SKAB, anomalies réelles étiquetées). Ce volet ne concerne pas les tôles : il est traité comme un second besoin de la même usine.

Le reste est à vous : les modèles, la façon de découper le problème (un classifieur puis un détecteur, un détecteur seul, autre chose), les métriques, les seuils, les données supplémentaires si vous en trouvez de publiques. Les dernières étapes des TD 1 à 4 et le TD 6 donnent des points de départ ; vous pouvez vous en écarter, à condition de le justifier. Si vous avez le temps, testez aussi la robustesse du système à des conditions dégradées (flou, éclairage, bruit) : ce n'est pas exigé, mais cela renforce une recommandation.

Ce que vous devez montrer

Quelle que soit votre approche, votre étude doit permettre à l'usine de vous faire confiance. Pour cela :

1. Une solution simple sert de référence : une méthode sans deep learning, réglée sérieusement. Le deep learning doit montrer ce qu'il apporte par rapport à elle.
2. Les mesures sont honnêtes : tous les choix se font sur une validation, et le test ne sert qu'une fois, à la fin. Les méthodes comparées le sont sur les mêmes données, avec la même métrique.
3. Les choix sont chiffrés. Vous fixez vous-mêmes des hypothèses sur ce que coûte un défaut non vu et une fausse alerte, vous les écrivez, et vos seuils en découlent.
4. Les limites sont dites : sur quels défauts, dans quelles conditions le système se trompe, et ce que cela coûterait.
5. Le travail se relance : un autre ingénieur doit pouvoir reproduire vos résultats à partir de votre dépôt.

Organisation

Groupes de 3 ou 4. Chaque groupe envoie sa composition à l'enseignant.

Rendu

À rendre deux semaines après le Jour 6, à 23 h 59, sur le dépôt privé du groupe (l'enseignant, FranklinB13, en collaborateur). C'est le dernier commit poussé avant cette heure qui est corrigé.

- Le code : les notebooks marimo du projet dans `projet/`, organisés comme vous le souhaitez (le gabarit `projet/brique_modele.py` peut servir de point de départ), avec un `projet/README.md` qui dit dans quel ordre les lancer, et les copies avec sorties enregistrées par marimo dans `projet/__marimo__/`.
- Les poids des modèles retenus et les résultats des calculs longs, sauvegardés dans le dépôt (dans `projet/`, et non dans `modeles/` ou `cache/`, qui ne sont pas versionnés). À partir de ces sauvegardes, l'ensemble des notebooks doit se relancer en moins de 15 minutes sur une machine de l'école. Les entraînements eux-mêmes restent dans le code, pour qu'on puisse les refaire.
- Un rapport de 10 pages au maximum, `projet/rapport.pdf`, écrit pour la direction de l'usine : le problème, votre solution, les résultats sur le test, les limites, et votre recommandation (déployer, déployer sous conditions, ou ne pas déployer), argumentée. Il se termine par une ligne qui indique si vous avez utilisé des outils d'IA, et pour quoi faire.
- Un historique Git régulier pendant tout le module.

Usage des outils d'IA

Utilisez le moins possible les outils d'IA générative (assistants de code, générateurs de texte), aussi bien pour le code que pour le rapport. Le projet sert à vérifier que vous savez mener cette étude vous-mêmes.

Le code doit être commenté : un lecteur doit comprendre ce que fait chaque partie et pourquoi. Le rapport doit être bien rédigé, avec vos propres mots. Une mise en page simple écrite par vous est préférable à un document soigné produit par une IA.

Notation

Une seule note pour le groupe, sur 20.

Critère	Points
Rigueur de la démarche	6
Pertinence des choix	5
Mise en production	3
Analyse critique et recommandation	4
Reproductibilité et qualité du code	2

Pour chaque critère, voici ce qui distingue une note faible, moyenne et haute.

Rigueur de la démarche (6 points) :

- Faible (0 à 2) : pas de référence sans deep learning, ou des réglages faits sur le test, ou des méthodes comparées sur des données différentes ;
- Moyenne (3 à 4) : référence et protocole présents, mais référence peu réglée ou comparaisons incomplètes ;
- Haute (5 à 6) : référence réglée avec soin, validation et test séparés pour tout le système et pour le volet au choix, test mesuré une seule fois, comparaisons équitables.

Pertinence des choix (5 points) :

- Faible (0 à 1) : choix non expliqués, métrique par défaut sans réflexion ;
- Moyenne (2 à 3) : choix expliqués, mais seuils ou hypothèses de coût peu reliés au besoin ;
- Haute (4 à 5) : découpage du problème, modèles et métriques justifiés par le besoin de l'usine, hypothèses de coût écrites et seuils qui en découlent, volet au choix traité avec la même exigence.

Mise en production (3 points) :

- Faible (0 à 1) : temps non mesuré, ou mesuré sans conditions précises ;
- Moyenne (2) : temps mesuré, optimisation appliquée, mais perte de qualité non mesurée ou plan de suivi vague ;
- Haute (3) : cadence vérifiée sur CPU, gain et perte de l'optimisation mesurés, plan de suivi concret avec des seuils.

Analyse critique et recommandation (4 points) :

- Faible (0 à 1) : résultats présentés sans analyse, recommandation absente ou non argumentée ;
- Moyenne (2 à 3) : erreurs commentées, recommandation présente mais peu reliée aux mesures ;
- Haute (4) : cas d'échec analysés et chiffrés, limites assumées, recommandation qui découle des mesures, rapport clair et bien rédigé.

Reproductibilité et qualité du code (2 points) :

- Faible (0) : le dépôt ne se relance pas ;
- Moyenne (1) : il se relance, mais avec des corrections, ou le code est peu commenté ;
- Haute (2) : il se relance tel quel en moins de 15 minutes, le code est commenté, l'historique montre un travail régulier.

Sont pénalisés dans tous les cas : un réglage fait sur le test, des chiffres qui ne viennent d'aucune exécution, un dépôt qui ne se relance pas. Un résultat modeste bien mesuré vaut mieux qu'un bon score mal mesuré.

Données et outils

NEU-DET est à télécharger sur Kaggle et à placer dans `donnees/NEU/` (README du dépôt, section Données). SKAB se télécharge avec `tp.charger_skab()`. Les fonctions de `outils/tp_utils.py` restent utilisables dans `projet/` ; les fonctions écrites pendant les TD ne s'importent pas, il faut les recopier.

Pour NEU-DET, utilisez un seul découpage pour tout le projet : `tp.decoupage_neu()` répartit les images en entraînement, validation et test, et `tp.charger_neu` (classification) comme `tp.voc_vers_yolo(..., parties=tp.decoupage_neu())` (détection) le respectent, à paramètres égaux. Les étapes 11 des TD 1 et 2 ne découpent pas les images de la même façon : ne réutilisez pas tel quel le jeu YOLO créé au TD 2.

Aide et indications

Quelques pièges fréquents, à vérifier avant de rendre :

- Une image ne doit jamais se trouver à la fois dans l'entraînement d'un modèle et dans le test d'un autre, si les deux modèles forment un même système.
- Réduire un type de défaut à 15 images ne concerne que l'entraînement : la validation et le test gardent toutes leurs images, sinon on ne peut plus mesurer ce que le système vaut sur ce défaut.
- Quand les types de défauts n'ont pas le même nombre d'images, une accuracy globale peut cacher un type jamais reconnu : regardez les résultats type par type.
- Sur CPU, un entraînement complet peut durer longtemps : mettez au point vos réglages sur un sous-ensemble ou à résolution réduite, lancez les entraînements longs tôt (le soir, pendant une pause), et sauvegardez les poids pour ne pas avoir à les refaire.
- Un temps d'inférence se mesure image par image, sur plusieurs dizaines d'images, sans compter le premier passage (plus lent), et en précisant la machine.
- Une optimisation qui accélère le modèle se juge aussi sur ce qu'elle lui fait perdre en qualité, mesuré sur les mêmes images.
- Pour SKAB, tout ce qui est calculé à partir des données (moyennes, écarts-types, seuils) se calcule sur la partie d'entraînement, puis s'applique tel quel au reste.
- Un seuil qui maximise le F1 n'est pas forcément le bon : demandez-vous ce que coûte chaque erreur.
- Avant de rendre, clonez votre dépôt dans un dossier vide et relancez tout : c'est ce que fera le correcteur.